Chapter 1

Factor graphs and belief propagation

The paper's central computational claim is that the score of a non-Gaussian, correlated law in L dimensions is available exactly, by "one forward–backward sweep of functional message passing". This chapter builds the objects in that sentence from scratch: what a factor graph is, what the sum-product algorithm does on it, why the algorithm is exact when the graph is a tree, and why conditioning the chain prior on coordinatewise noised observations leaves a tree. It then specialises the general algorithm to the two message sweeps of the paper's equation for $\overrightarrow{m}_i$ and $\overleftarrow{m}_i$ (Appendix B, the equation labelled *bp*), reads off the site marginals, the pairwise marginals and the evidence, and places the recursion next to its two classical relatives, the forward–backward algorithm for hidden Markov models and the Kalman smoother.

1.1 Graphical models and factor graphs

1.1.1 Factorised functions

Let $a = (a_1, \dots, a_L)$ be a vector of variables, each taking values in a set $\mathcal{A}$ (a finite alphabet, or $\mathbb{R}$). A **factorisation** of a non-negative function f of a is a representation

$$f(a) = \prod_{\alpha \in \mathcal{F}} \psi_\alpha(a_{\partial\alpha}), \tag{1.1}$$

where $\mathcal{F}$ is a finite index set of **factors**, each factor α has a **scope** $\partial\alpha \subseteq \{1, \dots, L\}$, and $a_{\partial\alpha}$ is the sub-vector of the variables in that scope. The **factor graph** of the factorisation is the bipartite graph with one **variable node** per index i, one **factor node** per factor α, and an edge between i and α whenever $i \in \partial\alpha$. A factor with a scope of size one is called **unary**; a scope of size two is **pairwise**.

When f is a probability density up to a constant, $p(a) = f(a)/Z$ with $Z = \int f(a)\,\mathrm{d}a$, the factor graph is a **graphical model** of p. The constant Z is the **normaliser**, also called the **partition function** in physics and, when f is a joint density of data and latent variables evaluated at the observed data, the **evidence** or **marginal likelihood**.

Example 1.1 (The chain prior)**.** The paper's clean law, its Section 2, equation *prior*, is $P_0(\mathbf{a}) = \mu(a_1)\prod_{i=2}^{L} K(a_i \mid a_{i-1})$. It is a factorisation with one unary factor μ of scope $\{1\}$ and $L-1$ pairwise factors $K(\cdot \mid \cdot)$ of scopes $\{i-1, i\}$. Its factor graph is a path: variable nodes $a_1, a_2, \dots, a_L$ with a factor node between each consecutive pair, and a dangling unary factor at a_1. Here $Z = 1$ because each $K(\cdot \mid a)$ integrates to one.

1.1.2 Conditional independence

The factor graph encodes which variables are conditionally independent of which. The statement we need is the following. If removing the variable node a_i (together with its edges) disconnects the

graph into components $\mathcal{C}_1, \dots, \mathcal{C}_r$, then under p the groups of variables in different components are conditionally independent given a_i. The proof is one line: every factor lies inside one component (its scope, minus i, is connected through the factor node), so $f(a) = \prod_j g_j(a_{\mathcal{C}_j}, a_i)$, and a joint density that is a product of functions of the separate groups (each also depending on a_i) is, given a_i, a product density.

Applied to the path of Example 1.1, removing a_i separates the past $\{a_1, \dots, a_{i-1}\}$ from the future $\{a_{i+1}, \dots, a_L\}$: this is the **Markov property**, and it is the only use we make of the word "Markov".

1.1.3 Trees, and why marginalisation is hard in general

A factor graph is a **tree** when it is connected and has no cycles. A path is a tree. The **marginal** of p at a variable i is $p_i(a_i) = \frac{1}{Z} \int f(a)\, \mathrm{d}a_{\setminus i}$, integrating over every variable but the i-th. Written out on a finite alphabet of size N_g, the integral is a sum over N_g^{L-1} configurations, which for the paper's $N_\mathrm{g} = 401$ and $L = 32$ is not a computation. Every algorithm in this chapter is a way of organising that sum so that it costs $O(LN_\mathrm{g}^2)$ instead, and the organisation is possible exactly because the graph is a tree.

1.2 The sum-product algorithm

1.2.1 Messages

The **sum-product algorithm**, also called **belief propagation**, passes two kinds of functions along the edges of the factor graph. For an edge between variable i and factor α:

$$\text{variable to factor:} \qquad m_{i\to\alpha}(a_i) = \prod_{\beta \in \partial i \setminus \{\alpha\}} m_{\beta\to i}(a_i), \tag{1.2}$$

$$\text{factor to variable:} \qquad m_{\alpha\to i}(a_i) = \int \psi_\alpha(a_{\partial\alpha}) \prod_{j \in \partial\alpha \setminus \{i\}} m_{j\to\alpha}(a_j)\, \mathrm{d}a_{\partial\alpha\setminus i}, \tag{1.3}$$

where ∂i is the set of factors whose scope contains i. An empty product is 1, so a leaf variable (one attached to a single factor) sends the constant message 1, and a leaf factor (a unary factor) sends its own function $m_{\alpha\to i}(a_i) = \psi_\alpha(a_i)$. The names are literal: a message is a function of the variable at the edge's variable end, and Equation (1.3) sums (integrates) a product.

On a tree, every message can be computed once and in a definite order: root the tree anywhere, compute messages from the leaves towards the root, then from the root back towards the leaves. Each edge carries one message in each direction, and each message is computed after all the messages it depends on. The **belief** at a variable and at a factor are then

$$b_i(a_i) \propto \prod_{\alpha \in \partial i} m_{\alpha\to i}(a_i), \qquad b_\alpha(a_{\partial\alpha}) \propto \psi_\alpha(a_{\partial\alpha}) \prod_{j \in \partial\alpha} m_{j\to\alpha}(a_j). \tag{1.4}$$

1.2.2 Exactness on a tree

Theorem 1.2 (Sum-product is exact on a tree)**.** *Let the factor graph of Equation* (1.1) *be a tree. Then the beliefs of Equation* (1.4) *are the exact marginals: $b_i = p_i$ and $b_\alpha = p_{\partial\alpha}$, where $p_{\partial\alpha}$ is the marginal of p on the scope of α. Moreover, for every variable i, $Z = \int \prod_{\alpha\in\partial i} m_{\alpha\to i}(a_i)\, \mathrm{d}a_i$.*

Proof. The proof is an induction on subtrees, and it rests on one fact about integrals of products: the **distributive law**, $\int g(u)h(v)\, \mathrm{d}u\, \mathrm{d}v = \int g\, \mathrm{d}u \int h\, \mathrm{d}v$ when the two factors depend on disjoint sets of variables.

Fix an edge (i, α) and delete it. Because the graph is a tree, this splits it into two subtrees; call $T_{\alpha\to i}$ the one containing α and $T_{i\to\alpha}$ the one containing i. Write $F_{\alpha\to i}(a_i)$ for the integral,

over every variable of $T_{\alpha\to i}$ other than a_i, of the product of all factors in $T_{\alpha\to i}$; and $F_{i\to\alpha}(a_i)$ likewise for $T_{i\to\alpha}$ (the factors on the i side other than α). We claim

$$m_{\alpha\to i} = F_{\alpha\to i}, \qquad m_{i\to\alpha} = F_{i\to\alpha}. \tag{1.5}$$

The claim is proved by induction on the number of nodes in the subtree behind the message. If $T_{\alpha\to i}$ consists of α alone, then α is unary and both sides equal $\psi_\alpha(a_i)$. If $T_{i\to\alpha}$ consists of i alone, both sides are 1. Otherwise, suppose the claim holds for all smaller subtrees. The subtree $T_{\alpha\to i}$ is α together with, for each $j \in \partial\alpha \setminus \{i\}$, the subtree $T_{j\to\alpha}$; these subtrees are disjoint (no cycles) and share no variables with one another (a shared variable would be a cycle through α). Hence the product of all factors in $T_{\alpha\to i}$ is $\psi_\alpha \prod_j (\text{factors of } T_{j\to\alpha})$, and integrating out the variables of each $T_{j\to\alpha}$ other than a_j can be done separately by the distributive law, giving $\psi_\alpha \prod_j F_{j\to\alpha}(a_j)$; the remaining integral over $a_{\partial\alpha\setminus i}$ is exactly Equation (1.3) once $F_{j\to\alpha} = m_{j\to\alpha}$ by the induction hypothesis. The variable-to-factor case is the same argument with the subtrees $T_{\beta\to i}$, $\beta \in \partial i \setminus \{\alpha\}$, which share only the variable a_i; no integral is needed and Equation (1.2) follows.

Now take a variable i. The whole graph is the union of the subtrees $T_{\alpha\to i}$, $\alpha \in \partial i$, which pairwise share only a_i. By the distributive law again, $\int f(a)\,\mathrm{d}a_{\setminus i} = \prod_{\alpha\in\partial i} F_{\alpha\to i}(a_i)$, and by Equation (1.5) this is the unnormalised belief. Integrating once more over a_i gives Z. For a factor α, the whole graph is α together with the subtrees $T_{j\to\alpha}$, $j \in \partial\alpha$, sharing no variables, and the same reasoning gives $b_\alpha = p_{\partial\alpha}$. □

Three remarks on the theorem. First, nothing in the proof depends on the variables being discrete: on $\mathbb{R}$ the messages are functions and the sums are integrals, which is what the paper means by *functional* message passing. Second, the proof shows that the belief at a factor integrates, over the variables of the factor other than i, to the belief at i: beliefs are **locally consistent**. Third, on a graph with cycles the messages of Equations (1.2) and (1.3) still make sense and can be iterated (**loopy belief propagation**), but the induction fails at the first cycle and the beliefs are approximations. The paper's claim is that its inference is *tree-exact*: the posterior graph is a tree, so no such approximation is made.

1.3 The chain posterior of the paper

1.3.1 Conditioning adds only unary factors

The observation model is the coordinatewise channel of the paper's equation *channel*: given the clean chain $\mathbf{a}$, the coordinates of $\mathbf{x} = e^{-t}\mathbf{a} + \sqrt{\Delta_t}\mathbf{z}$ are independent, each with density $\mathcal{N}(e^{-t}a_i, \Delta_t)$ in x_i. By Bayes' rule,

$$p_t(\mathbf{a} \mid \mathbf{x}) = \frac{P_0(\mathbf{a}) \prod_{i=1}^{L} \ell_t(x_i \mid a_i)}{P_t(\mathbf{x})}, \qquad \ell_t(x_i \mid a_i) = \frac{1}{\sqrt{2\pi\Delta_t}} \exp\Big[-\frac{(x_i - e^{-t}a_i)^2}{2\Delta_t}\Big]. \tag{1.6}$$

Compared with the prior, the numerator has one extra factor per site, each of scope $\{i\}$: a **unary likelihood factor**. No factor couples two latent variables that were not already coupled. The factor graph of the posterior is therefore the prior's path with a unary factor hanging from every variable node; it is still a tree (Figure 1.1). This is the whole content of the sentence "conditioning leaves the posterior factor graph a tree". It would fail for a channel that mixed coordinates (a non-diagonal noise covariance would add pairwise factors among the a_i, closing cycles), and it holds for any prior whose graph is a tree, not only for a path.

1.3.2 From the general rules to the two sweeps

Specialise Equations (1.2) and (1.3) to Figure 1.1. Each latent variable a_i has at most three neighbouring factors: the transition factor on its left, the transition factor on its right, and

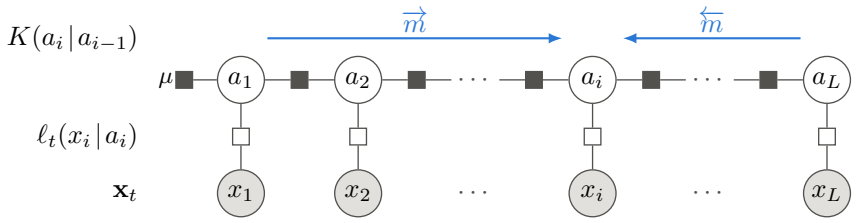


Figure 1.1: The factor graph of the chain posterior, drawn as in the paper's Figure 1. Latent sites (white circles) are joined by the transition factors K (filled squares) and, at the left end, by the initial factor μ. Conditioning attaches one unary likelihood factor (open square) per site, joining a_i to its own observation x_i (shaded circle), and adds no edge among the latent variables. The graph is a tree, so the forward sweep $\overrightarrow{m}$ and the backward sweep $\overleftarrow{m}$ of the sum-product algorithm are exact; they meet at the query site i.

its unary likelihood. Write K_i for the factor between a_{i-1} and a_i (with $K_1 = \mu$ the unary initial factor). Going left to right, the message that K_i sends into a_i is, by Equation (1.3), an integral over a_{i-1} of $K(a_i \mid a_{i-1})$ times the message a_{i-1} sends into K_i; and that message is, by Equation (1.2), the product of what a_{i-1} receives from its other two neighbours, its own unary factor $\ell_t(x_{i-1} \mid \cdot)$ and the transition factor on its left. Naming $\overrightarrow{m}_i(a) = m_{K_i \to a_i}(a)$, this reads

$$\overrightarrow{m}_1(a) = \mu(a), \qquad \overrightarrow{m}_i(a) = \int K(a \mid a')\, \ell_t(x_{i-1} \mid a')\, \overrightarrow{m}_{i-1}(a')\, \mathrm{d}a', \quad i \geq 2. \tag{1.7}$$

Going right to left, name $\overleftarrow{m}_i(a) = m_{K_{i+1} \to a_i}(a)$ the message that the transition factor on the right of a_i sends into a_i; for the last site there is no such factor and $\overleftarrow{m}_L \equiv 1$ (an empty product). The same two rules give

$$\overleftarrow{m}_L(a) = 1, \qquad \overleftarrow{m}_i(a) = \int K(a'' \mid a)\, \ell_t(x_{i+1} \mid a'')\, \overleftarrow{m}_{i+1}(a'')\, \mathrm{d}a'', \quad i \leq L-1. \tag{1.8}$$

These are, up to the proportionality signs discussed in Section 1.3.5, the two lines of the paper's equation *bp*. The paper's convention for $\overrightarrow{m}_1$ differs from Equation (1.7) in one cosmetic respect: it writes $\overrightarrow{m}_1 \equiv 1$ and treats μ as if it were part of the first site's unary factor. The code follows Equation (1.7), with `L[0] = exp(log_mu)` and `R[-1] = 1` in `src/markov_score/bp_grid.py`. Note the asymmetry of the kernel's arguments: forward, the integration variable is the *conditioning* argument of K; backward, it is the *conditioned* one. The transition matrix on a grid, $K[k,l] = K(u_k \mid u_l)$ with column index the parent, enters the forward sweep as K and the backward sweep as $K^\top$, which is exactly what the code does.

Two conventions are worth fixing, because both appear in the literature. The messages Equations (1.7) and (1.8) *exclude* the unary factor of the site they arrive at; they are the factor-to-variable messages of the general algorithm. The alternative convention (the "filtered" forward variable of the HMM literature) multiplies the local likelihood in and would be $\overrightarrow{m}_i \ell_t$. The paper and the code use the excluded form, so that the local factor appears once, explicitly, in every formula below.

1.3.3 Site marginals, pairwise marginals and the evidence

By Equation (1.4), the belief at site i is the product of what it receives from its three neighbours,

$$p_t(a_i \mid \mathbf{x}) = \frac{\overrightarrow{m}_i(a_i)\, \ell_t(x_i \mid a_i)\, \overleftarrow{m}_i(a_i)}{Z_i}, \qquad Z_i = \int \overrightarrow{m}_i \ell_t(x_i \mid \cdot) \overleftarrow{m}_i \, \mathrm{d}a_i, \tag{1.9}$$

and by Theorem 1.2 it is the exact posterior marginal. The paper's phrase "site marginals $\propto \overrightarrow{m}_i \ell_t \overleftarrow{m}_i$" is this line. The belief at the transition factor between a_{i-1} and a_i is

$$p_t(a_{i-1}, a_i \mid \mathbf{x}) = \frac{\overrightarrow{m}_{i-1}(a_{i-1})\, \ell_t(x_{i-1} \mid a_{i-1})\, K(a_i \mid a_{i-1})\, \ell_t(x_i \mid a_i)\, \overleftarrow{m}_i(a_i)}{Z_{i-1,i}}, \tag{1.10}$$

the exact **pairwise marginal**, whose two single-site margins are Equation (1.9) at $i-1$ and at i (local consistency). The pairwise marginals are what the expectation–maximisation chapter (**??**) sums into the expected transition statistic $\boldsymbol{\Xi}$; they come from the same two sweeps at no extra cost beyond the outer products.

The normaliser is available at any site, $Z_i = P_t(\mathbf{x})$ for every i, because every Z_i equals $\int P_0(\mathbf{a}) \prod_j \ell_t(x_j \mid a_j)\,\mathrm{d}\mathbf{a}$ by the proof of Theorem 1.2; the same holds for every $Z_{i-1,i}$. In particular the forward sweep alone gives it, as $Z_L = \int \overrightarrow{m}_L \ell_t(x_L \mid \cdot)\,\mathrm{d}a$ once $\overleftarrow{m}_L \equiv 1$. This number, $\log P_t(\mathbf{x})$, is the **log-evidence**: the marginal log-likelihood of one noised sequence under the chain prior, which the paper's Section 4 maximises over the parameters of the kernel and which its Appendix E uses, summed over a held-out bundle, as an oracle-free model-selection criterion.

1.3.4 From the marginal to the score

The paper needs one number per site, the posterior mean $\langle a_i \rangle_{x,t} = \int a_i\, p_t(a_i \mid \mathbf{x})\,\mathrm{d}a_i$, because Tweedie's identity (the chapter on the Ornstein–Uhlenbeck channel) converts it into the score, $S(\mathbf{x},t)_i = -(x_i - e^{-t}\langle a_i \rangle_{x,t})/\Delta_t$. Thus the whole of "the exact score" is: two sweeps, L one-dimensional integrals for the means, and one subtraction per site. The word "exact" refers to Theorem 1.2; what is not exact is the numerical representation of the messages, which is the subject of **??**.

1.3.5 Normalisation and the log-normaliser

Messages are defined up to a positive constant: multiplying $\overrightarrow{m}_i$ by c multiplies every later forward message and every belief involving it by c, and the normalised beliefs Equations (1.9) and (1.10) are unchanged. This is why the paper writes $\propto$ in its recursion. Numerically, unnormalised messages on a long chain underflow or overflow (each step multiplies by a likelihood of order $1/\sqrt{2\pi\Delta_t}$ and by a kernel of order $1/\sqrt{q}$), so an implementation rescales each message to unit mass and keeps the discarded constants:

$$\widetilde{\overrightarrow{m}}_i = \frac{\overrightarrow{m}_i}{c_i}, \qquad c_i = \int \overrightarrow{m}_i\,\mathrm{d}a, \qquad \log P_t(\mathbf{x}) = \sum_{i=2}^{L} \log c_i + \log \int \widetilde{\overrightarrow{m}}_L \ell_t(x_L \mid \cdot)\,\mathrm{d}a, \tag{1.11}$$

where each c_i is computed from the *already rescaled* previous message, so that the product of the c_i is the total constant removed. The log-evidence is thus assembled from a sum of logarithms and never as a product of small numbers. **??** gives the grid version, including the constants of the likelihood factors that the implementation removes for stability and restores at the end.

1.3.6 Cost

On a grid of N_g points, a message is a vector of length N_g and a transition integral is a matrix–vector product with the $N_\mathrm{g} \times N_\mathrm{g}$ matrix $K[k,l]$, costing $O(N_\mathrm{g}^2)$. The forward sweep performs $L-1$ of them, the backward sweep the same, and the beliefs cost $O(LN_\mathrm{g})$. The total is $O(LN_\mathrm{g}^2)$ per sequence and per noise level, as the paper states in its conclusion, and it is linear in the chain length: this is why the length-extrapolation study of the paper's Section 5 costs nothing extra at $L = 256$. For a batch of B sequences the matrix–vector products become matrix–matrix products $(N_\mathrm{g} \times N_\mathrm{g})(N_\mathrm{g} \times B)$, which is how `grid_bp_batch` in `src/markov_score/bp_grid.py` organises the computation.

1.4 Three names for one recursion

Forward–backward for hidden Markov models. When the state space is a finite set of N_g symbols, Equations (1.7) and (1.8) are the forward and backward recursions of Baum

and colleagues (1970) and Rabiner's tutorial (1989), usually written for the filtered variable $\alpha_i(k) = P(a_i = k, x_1, \dots, x_i)$ and the backward variable $\beta_i(k) = P(x_{i+1}, \dots, x_L \mid a_i = k)$. In our notation $\alpha_i = \overrightarrow{m}_i \ell_t(x_i \mid \cdot)$ and $\beta_i = \overleftarrow{m}_i$; the posterior at a site is $\alpha_i \beta_i / P_t(\mathbf{x})$ and the pairwise posterior is $\alpha_{i-1} K \ell_t \beta_i / P_t(\mathbf{x})$, both as above. The pairwise posteriors summed over sites are the "expected transition counts" of the Baum–Welch algorithm, and their continuous analogue on the grid is exactly the paper's $\boldsymbol{\Xi}$. The paper's recursion is therefore not a new algorithm; the paper says so in its related work, citing Baum and Rabiner.

The Kalman smoother. When μ, K and ℓ_t are all Gaussian, every message is Gaussian (a product of Gaussians is Gaussian, and the convolution of a Gaussian with a Gaussian kernel is Gaussian), so each message is carried by two numbers, a mean and a variance, or equivalently a precision and an information vector. The forward sweep is then the Kalman filter and the combination of forward and backward messages is the Rauch–Tung–Striebel smoother. **??** writes out the two moment maps and proves the paper's Proposition 1, which says that projecting every message onto a Gaussian, for *any* innovation law with the same variance, reproduces this smoother. The implementation of the Gaussian case is `gaussian_chain_bp` in `src/markov_score/bp_gaussian.py`.

Functional message passing and Kitagawa's grid smoother. When the state is continuous and non-Gaussian, the messages are genuine functions on $\mathbb{R}$ and have no finite parametrisation. The paper calls the recursion "functional message passing" to stress that the algorithm is defined at the level of functions, before any discretisation, and "tree-exact inference" to stress that the only approximation is in representing those functions. Representing them by their values on a grid and replacing integrals by quadrature is Kitagawa's non-Gaussian smoother (1987), which the paper also cites. The phrase "the score" throughout the paper means: the posterior mean obtained by this recursion, evaluated on a grid whose adequacy is measured (**??**), inserted into Tweedie's identity.

1.5 A worked example: three sites, three states

Take $L = 3$ sites and a three-point alphabet $\{u_1, u_2, u_3\} = \{-1, 0, 1\}$ with no quadrature weights (a finite-state model, so that every integral is a sum). Let the initial law, the transition matrix and the three likelihood rows be

$$\mu = \begin{pmatrix} 0.25 \\ 0.5 \\ 0.25 \end{pmatrix}, \qquad K = \begin{pmatrix} 0.6 & 0.2 & 0.1 \\ 0.3 & 0.6 & 0.3 \\ 0.1 & 0.2 & 0.6 \end{pmatrix}, \qquad \ell_1 = \begin{pmatrix} 0.2 \\ 0.5 \\ 0.3 \end{pmatrix}, \ \ell_2 = \begin{pmatrix} 0.1 \\ 0.3 \\ 0.6 \end{pmatrix}, \ \ell_3 = \begin{pmatrix} 0.4 \\ 0.4 \\ 0.2 \end{pmatrix},$$

where $K[k, l]$ is the probability of moving from state l (column) to state k (row), so the columns sum to one, and $\ell_i[k]$ (the unary factor $\ell_t(x_i \mid u_k)$ of the chapter, written as a plain vector here) is the likelihood of the i-th observation given $a_i = u_k$ (any positive numbers; they are not normalised over k and need not be).

Forward sweep. $\overrightarrow{m}_1 = \mu$. The message into site 2 is $\overrightarrow{m}_2 = K(\overrightarrow{m}_1 \circ \ell_1)$, with $\circ$ the elementwise product: $\overrightarrow{m}_1 \circ \ell_1 = (0.05, 0.25, 0.075)$, and

$$\overrightarrow{m}_2 = \begin{pmatrix} 0.6 \cdot 0.05 + 0.2 \cdot 0.25 + 0.1 \cdot 0.075 \\ 0.3 \cdot 0.05 + 0.6 \cdot 0.25 + 0.3 \cdot 0.075 \\ 0.1 \cdot 0.05 + 0.2 \cdot 0.25 + 0.6 \cdot 0.075 \end{pmatrix} = \begin{pmatrix} 0.0875 \\ 0.1875 \\ 0.1 \end{pmatrix}.$$

Then $\overrightarrow{m}_2 \circ \ell_2 = (0.00875, 0.05625, 0.06)$ and $\overrightarrow{m}_3 = K(\overrightarrow{m}_2 \circ \ell_2) = (0.0225, 0.054375, 0.048125)$.

Backward sweep. $\overleftarrow{m}_3 = (1, 1, 1)$. Then $\overleftarrow{m}_2 = K^\top(\overleftarrow{m}_3 \circ \ell_3) = K^\top(0.4, 0.4, 0.2)$: the first entry is $0.6 \cdot 0.4 + 0.3 \cdot 0.4 + 0.1 \cdot 0.2 = 0.38$, and altogether $\overleftarrow{m}_2 = (0.38, 0.36, 0.28)$. Next $\overleftarrow{m}_2 \circ \ell_2 = (0.038, 0.108, 0.168)$ and $\overleftarrow{m}_1 = K^\top(\overleftarrow{m}_2 \circ \ell_2) = (0.072, 0.106, 0.137)$.

Evidence. From the end of the forward sweep, $Z = \sum_k \overrightarrow{m}_3[k]\ell_3[k] = 0.0225 \cdot 0.4 + 0.054375 \cdot 0.4 + 0.048125 \cdot 0.2 = 0.040375$. The same number is obtained at site 1, $\sum_k \mu[k]\ell_1[k]\overleftarrow{m}_1[k] = 0.25 \cdot 0.2 \cdot 0.072 + 0.5 \cdot 0.5 \cdot 0.106 + 0.25 \cdot 0.3 \cdot 0.137 = 0.040375$, and at site 2, as Theorem 1.2 requires.

Site marginals. At site 1 the unnormalised belief is $\mu \circ \ell_1 \circ \overleftarrow{m}_1 = (0.0036,\ 0.0265,\ 0.010275)$; dividing by Z gives $p(a_1 \mid \mathbf{x}) = (0.0892,\ 0.6564,\ 0.2545)$ and posterior mean $-0.0892 + 0.2545 = 0.1653$. At site 2 the unnormalised belief is

$$\overrightarrow{m}_2 \circ \ell_2 \circ \overleftarrow{m}_2 = (0.003325,\ 0.02025,\ 0.0168),$$

so that the normalised marginal is $p(a_2 \mid \mathbf{x}) = (0.0824,\ 0.5015,\ 0.4161)$, with mean 0.3337. At site 3 the same steps give $p(a_3 \mid \mathbf{x}) = (0.2229,\ 0.5387,\ 0.2384)$ and the mean 0.0155. Enumerating all 27 configurations of (a_1, a_2, a_3) and summing gives the same Z and the same three marginals, which is the finite version of Theorem 1.2.

Pairwise marginal. On the edge $(1, 2)$, Equation (1.10) gives the matrix with entries

$$p(a_2 = u_k, a_1 = u_l \mid \mathbf{x}) = \overleftarrow{m}_2[k]\,\ell_2[k]\,K[k,l]\,\overrightarrow{m}_1[l]\,\ell_1[l]/Z;$$

for instance the entry $(k, l) = (2, 2)$, staying at 0, is $0.108 \cdot 0.6 \cdot 0.25/0.040375 = 0.4012$, and the full matrix is

$$p(a_2 = u_k, a_1 = u_l \mid \mathbf{x}) = \begin{pmatrix} 0.0282 & 0.0471 & 0.0071 \\ 0.0401 & 0.4012 & 0.0602 \\ 0.0208 & 0.2081 & 0.1872 \end{pmatrix},$$

whose column sums are the site-1 marginal and whose row sums are the site-2 marginal, to the rounding shown. This is the object that **??** accumulates into $\boldsymbol{\Xi}$: summing the two edge matrices of this chain gives a 3×3 matrix of total mass 2, the number of transitions.

Where it is used. The paper, Section 2 ("the score is a posterior mean, and the posterior is a tree", Figure 1) and Appendix B (the recursion *bp*); Section 4 (the pairwise marginals behind $\boldsymbol{\Xi}$); Appendix E (the log-evidence as a selection criterion, E20). Code: `src/markov_score/bp_grid.py` (`grid_bp` and `grid_bp_batch`), `src/markov_score/em.py` (`e_step`, which adds the pairwise accumulation to the same sweeps), `src/markov_score/bp_gaussian.py` (the Gaussian special case), and `paper/figures/fig_mechanism.tex` (the figure mirrored above). Tests: the brute-force enumeration test in `tests/test_em_bp.py`; `tests/test_fisher_bruteforce.py`, which checks the evidence and the pairwise marginals against enumeration on a five-state, four-site chain; and `tests/test_gaussian_bp_equivalence.py`. Experiments: every registered experiment, E01 to E21, scores against this recursion.

References. J. Pearl (1988), *Probabilistic Reasoning in Intelligent Systems*, Morgan Kaufmann. F. R. Kschischang, B. J. Frey and H.-A. Loeliger (2001), "Factor graphs and the sum-product algorithm", *IEEE Transactions on Information Theory* 47(2). L. E. Baum, T. Petrie, G. Soules and N. Weiss (1970), "A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains", *Annals of Mathematical Statistics* 41(1). L. R. Rabiner (1989), "A tutorial on hidden Markov models and selected applications in speech recognition", *Proceedings of the IEEE* 77(2). G. Kitagawa (1987), "Non-Gaussian state-space modeling of nonstationary time series", *Journal of the American Statistical Association* 82(400). M. Mézard and A. Montanari (2009), *Information, Physics, and Computation*, Oxford University Press, Chapter 14.